

1074. Number of Submatrices That Sum to Target

Hard · Array · Hash Table · Matrix · Prefix Sum

Approach — 2D Prefix Sum + Subarray Sum Equals K

The solution combines two classic techniques: **2D prefix sums** to efficiently compute any rectangular submatrix sum in $O(1)$, and the **subarray sum equals K** hashmap trick to count how many of those rectangular sums equal the target.

In the first pass, the matrix is transformed in-place into a prefix sum table using inclusion-exclusion. In the second pass, every row pair (rl , rh) is fixed, reducing the 2D problem to a 1D column scan solved with a hashmap in $O(m)$.

Key Insight: For a fixed row range, the column prefix sum `prefSum` at column c represents the total of the submatrix from column 0 to c . Looking up `prefSum - target` in the hashmap reveals how many earlier columns produce a submatrix summing exactly to target.

Step-by-step Breakdown

Step 1 — Build the 2D prefix sum table

Iterate every cell (i , j) and update it to the cumulative sum of all elements in the rectangle from (0,0) to (i , j). Inclusion-exclusion formula: add the cell above and the cell to the left, then subtract the diagonal overlap, each with a boundary check.

Step 2 — Fix a row range [rl , rh]

Two nested loops pick every pair of rows rl (low) and rh (high), defining a horizontal band. All submatrices whose top and bottom rows fall within this band are counted in the next step.

Step 3 — Apply the subarray sum equals K trick

For each column c , compute `prefSum` — the band sum from column 0 to c . Initialize `hashMap[0] = 1` to handle submatrices starting at column 0. If `prefSum - target` exists in the hashmap, add its count to the answer, then record the current prefix sum.

Step 4 — Reset and repeat

After scanning all columns for a row pair, `hashMap.clear()` resets the state for the next pair. All valid submatrix counts are accumulated in `ans` and returned.

Solution Code

```
1  class Solution {
2  public:
3      int numSubmatrixSumTarget(vector<vector<int>>& matrix, int target) {
4          int n = matrix.size();
5          int m = matrix[0].size();
6          for(int i = 0; i < n; i++)
7          {
8              for(int j = 0; j < m; j++)
9              {
10                 if(i-1 >= 0) matrix[i][j] += matrix[i-1][j];
11                 if(j-1 >= 0) matrix[i][j] += matrix[i][j-1];
12                 if(i-1 >= 0 && j-1 >= 0) matrix[i][j] -= matrix[i-1][j-1];
13             }
14         }
15
16         unordered_map<int, int> hashMap;
17         int ans = 0;
18         for(int rl = 0; rl < n; rl++)
19         {
20             for(int rh = rl; rh < n; rh++)
21             {
22                 hashMap[0] = 1;
23                 for(int c = 0; c < m; c++)
24                 {
25                     int prefSum = matrix[rh][c];
26                     if(rl-1 >= 0) prefSum -= matrix[rl-1][c];
27                     if(hashMap.count(prefSum-target)) ans += hashMap[prefSum-
target];
28                     hashMap[prefSum]++;
29                 }
30                 hashMap.clear();
31             }
        }
```

```
32         }
33         return ans;
34     }
35 };
```

Complexity Analysis

TIME COMPLEXITY	SPACE COMPLEXITY
$O(n^2 \times m)$	$O(m)$
Two nested row-pair loops give $O(n^2)$. Each pair does a single $O(m)$ column scan. Total: $O(n^2 \cdot m)$.	The hashmap holds at most $O(m)$ prefix sums at any time. The matrix is modified in-place, so no extra $O(n \cdot m)$ space is used.